Análise Comparativa de Estratégias de Controle Populacional em Algoritmos Genéticos Aplicados ao Problema das N-Rainhas

Vinícius Rubin[1], **Igor Yepes**[1]

[1]Instituto Federal de Educação, Ciência e Tecnologia Farroupilha – Campus Frederico Westphalen (IFFAR-FW)

Abstract. *This article investigates how different population control strategies affect the performance of genetic algorithms applied to the n-queens problem. The study presents the design of a baseline implementation, details the experimental protocol, and discusses the interpretation of metrics such as execution time, number of generations, and success rate. The objective is to identify configurations capable of preserving population diversity while accelerating convergence, offering practical guidelines for applying evolutionary heuristics to combinatorial search problems.*
Keywords: *Genetic Algorithms, Population Control, Evolutionary Computation, N-Queens Problem.*

Resumo. *Este artigo investiga como diferentes estratégias de controle populacional influenciam o desempenho de algoritmos genéticos aplicados ao problema das n-rainhas. São apresentados o planejamento da implementação base, o protocolo experimental e a análise de métricas como tempo de execução, número de gerações e taxa de sucesso. O objetivo é identificar configurações capazes de preservar a diversidade populacional e, ao mesmo tempo, acelerar a convergência, fornecendo orientações práticas para o uso de heurísticas evolutivas em tarefas de busca combinatória.*
Palavras-chave: *Algoritmos Genéticos, Controle Populacional, Computação Evolutiva, Problema das N-Rainhas.*

1. Introdução

Algoritmos Genéticos (AGs) são métodos de busca inspirados nos mecanismos de evolução natural, amplamente empregados na resolução de problemas complexos devido à sua capacidade de equilibrar exploração estocástica e intensificação por meio de operadores simples de reprodução [Eiben and Smith 2015, Holland 1975]. Esses algoritmos têm sido aplicados em diversas áreas — de problemas combinatórios clássicos a sistemas de engenharia — e sua eficiência depende de um conjunto de decisões de projeto que afetam diretamente o processo evolutivo.

Entre esses componentes, o controle populacional destaca-se como um dos fatores menos padronizados e, ao mesmo tempo, mais influentes no comportamento dos AGs. Esse mecanismo determina como indivíduos são preservados, substituídos ou eliminados ao longo das gerações, afetando diversidade, pressão seletiva, estabilidade evolutiva e risco de convergência prematura. Embora estratégias como substituição total, substituição parcial, elitismo e técnicas baseadas em ciclos de vida sejam amplamente relatadas na

literatura, comparações diretas entre elas são raras. Os estudos existentes geralmente variam simultaneamente representação, operadores ou configurações experimentais, o que dificulta isolar o impacto específico da política de substituição.

Essa ausência de padronização configura uma lacuna importante na área: ainda não está claro, sob condições controladas, como diferentes estratégias de controle populacional influenciam taxa de sucesso, custo computacional e profundidade de busca em AGs. Assim, surge a seguinte pergunta de pesquisa:

> *Como diferentes estratégias de controle populacional afetam o desempenho de Algoritmos Genéticos quando todos os demais componentes do algoritmo permanecem constantes?*

Para investigar essa questão, este trabalho realiza uma comparação sistemática entre quatro mecanismos de controle populacional — substituição total, substituição parcial, elitismo e uma abordagem baseada em idade inspirada no modelo GAVaPS — mantendo fixos representação, função de aptidão, seleção, cruzamento e mutação. O problema das n-rainhas é adotado como cenário experimental por constituir um benchmark clássico, escalável e amplamente utilizado na avaliação de heurísticas evolutivas.

A contribuição principal deste estudo consiste em fornecer uma análise empírica rigorosa e reprodutível sobre o efeito de diferentes políticas de substituição na dinâmica evolutiva dos AGs. Especificamente, o trabalho:

1. compara, sob condições uniformes, o desempenho de quatro estratégias de controle populacional;
2. avalia como cada mecanismo influencia taxa de sucesso, número de gerações, tempo de execução e limites de parada;
3. discute vantagens, limitações e cenários de aplicação de cada abordagem.

Os experimentos incluem múltiplas execuções independentes por configuração e utilizam critérios híbridos de parada (tempo, geração máxima e solução encontrada), possibilitando medir médias, variâncias e padrões de comportamento evolutivo. Essa estrutura fornece evidências quantitativas que auxiliam na escolha da estratégia de controle populacional mais adequada para problemas combinatórios de natureza semelhante.

2. Fundamentação Teórica

A fundamentação teórica deste trabalho aborda os conceitos essenciais para compreender o funcionamento dos Algoritmos Genéticos (AGs) e sua aplicação ao problema das n-rainhas, bem como os principais elementos estruturais que orientam a condução dos experimentos apresentados. Inicialmente, são discutidos os princípios básicos dos AGs, destacando-se sua inspiração biológica e os componentes fundamentais que determinam seu comportamento evolutivo. Em seguida, apresenta-se o fluxo geral de execução de um AG clássico, incluindo um pseudocódigo que servirá de referência para todas as variantes analisadas ao longo deste estudo.

O problema das n-rainhas é então revisitado como cenário experimental amplamente utilizado na literatura devido à sua natureza combinatória e à capacidade de parametrização da dificuldade. Posteriormente, descreve-se o ambiente computacional adotado — baseado na linguagem Python e em bibliotecas numéricas — utilizado para garan-

tir reprodutibilidade e eficiência nas análises. Por fim, detalham-se as estratégias de controle populacional investigadas, incluindo abordagens clássicas (substituição total, substituição parcial e elitismo) e uma técnica inspirada no modelo GAVaPS, que introduz controle por idade como forma de manutenção e renovação da população. Essa organização fornece a base conceitual necessária para compreender a motivação, execução e análise dos experimentos desenvolvidos ao longo do trabalho.

2.1. Algoritmos Genéticos

Algoritmos Genéticos (AGs) são meta-heurísticas inspiradas em princípios de evolução biológica, nas quais uma população de soluções candidatas é iterativamente modificada por operadores de seleção, cruzamento e mutação [Eiben and Smith 2015, Holland 1975]. Cada indivíduo é avaliado por uma função de aptidão (*fitness*), e o processo evolutivo tende a favorecer configurações de melhor desempenho ao longo das gerações.

O comportamento de um AG é fortemente influenciado por três grupos de decisões: (i) representação dos indivíduos, (ii) função de avaliação e (iii) operadores evolutivos e política de substituição [de Lacerda and de Leon Ferreira de Carvalho 1999].

Enquanto representação e operadores de cruzamento/mutação são amplamente discutidos na literatura, o *controle populacional* — isto é, a forma como indivíduos são mantidos, substituídos, removidos ou preservados — recebe tratamento menos sistemático, embora influencie diretamente diversidade, convergência e risco de aprisionamento em ótimos locais.

Neste trabalho, representação, função de aptidão, seleção, cruzamento e mutação permanecem fixos. O foco recai exclusivamente sobre o componente de controle populacional, permitindo isolar e mensurar o impacto de diferentes políticas de substituição no desempenho geral do algoritmo.

2.2. Fluxo Geral de um Algoritmo Genético

Embora diferentes variantes existam, o fluxo de execução de um AG tradicional segue uma estrutura geral bem definida. O pseudocódigo abaixo resume esse processo:

Algorithm 1 Pseudocódigo de um Algoritmo Genético Clássico

Entrada: tamanho populacional N, número máximo de gerações $G_{\max}$, operadores genéticos
Saída: melhor solução encontrada

Inicializar população P com N indivíduos
Avaliar P
for $g = 1$ até $G_{\max}$ **do**
// Seleção e Reprodução
    Selecionar pais usando torneio (ou outro método)
    Aplicar cruzamento para gerar descendentes
    Aplicar mutação aos descendentes
    Avaliar descendentes
// Controle Populacional
    Combinar populações conforme a estratégia escolhida:
    – substituição total, ou
    – substituição parcial, ou
    – elitismo, ou
    – controle por idade (GAVaPS/Michalewicz)
    Atualizar população P
end for
Retornar melhor indivíduo de P

Esse pseudocódigo será utilizado como base para todas as variações testadas neste estudo, modificando-se apenas a estratégia de controle populacional aplicada em cada experimento.

2.3. O Problema das *n*-Rainhas

O problema das *n*-rainhas consiste em posicionar *n* rainhas em um tabuleiro de xadrez de dimensões *n*×*n* de forma que nenhuma delas ataque outra, o que implica ausência de conflitos em linhas, colunas e diagonais [Bell and Stevens 2009]. Embora a formulação seja simples, o número de configurações possíveis cresce rapidamente com *n*, tornando abordagens exaustivas inviáveis para instâncias maiores.

Estudos como o de Ali et al. [Ali et al. 2023] mostram que buscas clássicas (BFS/DFS) são eficientes apenas para instâncias pequenas, tornando o problema ideal para testar meta-heurísticas devido ao equilíbrio entre dificuldade controlável e estrutura combinatória clara. O problema é amplamente utilizado como *benchmark* em estudos sobre AGs [Sarkar and Nag 2017, Farhan et al. 2015], permitindo comparação direta com trabalhos da literatura.

No presente estudo, utilizamos o problema das *n*-rainhas como cenário experimental para avaliar diferentes estratégias de controle populacional.

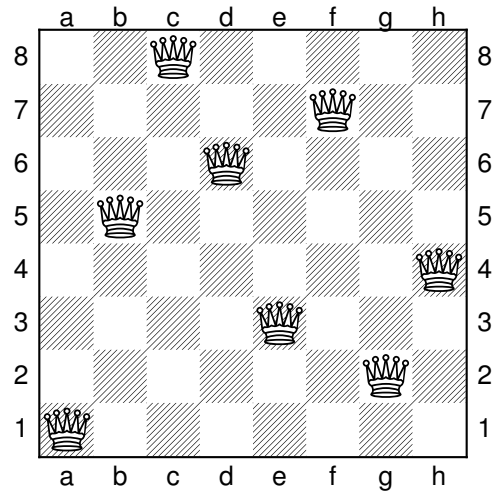


Figura 1. Exemplo de solução válida para o problema das 8-rainhas.

2.4. Uso da Linguagem Python e Bibliotecas Relacionadas

A implementação experimental foi desenvolvida em `Python`, amplamente adotado em computação científica pela clareza sintática e pelo ecossistema rico em bibliotecas numéricas [Python Software Foundation 2025a].

A biblioteca `NumPy` foi utilizada para operações vetorizadas e representação da população [Harris et al. 2020]. A geração de números aleatórios baseou-se em `numpy.random` [Python Software Foundation 2025b], e a visualização dos resultados foi realizada com `matplotlib` [Hunter 2007].

Esse conjunto permite foco total na experimentação e análise do controle populacional, evitando que detalhes de infraestrutura influenciem nos resultados.

2.5. Técnicas de Controle Populacional Abordadas Neste Trabalho

Este artigo compara quatro estratégias de controle populacional aplicadas sobre o mesmo AG base, mantendo fixos representação, função de aptidão, método de seleção, cruzamento e mutação. Dessa forma, torna-se possível isolar o efeito da política de substituição sobre a dinâmica evolutiva, em particular sobre diversidade, convergência e risco de aprisionamento em ótimos locais [Holland 1975, Goldberg 1989, Eiben and Smith 2015, Whitley 1994]. As estratégias avaliadas diferem principalmente em dois aspectos: (i) a fração da população que é renovada em cada geração e (ii) o critério utilizado para decidir quais indivíduos permanecem ou são removidos [de Lacerda and de Leon Ferreira de Carvalho 1999, Mitchell 1996].

2.5.1. Substituição Total

Na substituição total, a cada geração todos os indivíduos da população corrente são substituídos pelos descendentes recém-gerados. Em termos operacionais, o algoritmo:

1. seleciona pares de pais na população atual;

2. aplica cruzamento e mutação para produzir uma nova população de tamanho N;
3. descarta integralmente a população anterior.

Trata-se do esquema mais simples e amplamente utilizado em AGs, pois facilita a implementação e o controle de parâmetros [Goldberg 1989, Whitley 1994]. No entanto, por descartar toda a geração anterior, tende a reduzir a diversidade rapidamente, tornando o algoritmo mais suscetível à convergência prematura, especialmente quando a pressão seletiva é alta ou a mutação é conservadora [De Jong 1975, Spears 1993]. Nesta pesquisa, a substituição total é utilizada como referência de comparação para as demais estratégias.

2.5.2. Substituição Parcial

Na substituição parcial, apenas uma fração da população é renovada a cada geração, caracterizando um modelo do tipo *steady-state* [Eiben and Smith 2015, Whitley 1994]. De forma geral, o processo segue os passos:

1. a partir da população atual, selecionam-se pais e produzem-se alguns descendentes;
2. esses descendentes são inseridos na população;
3. um subconjunto de indivíduos aleatórios é removido, preservando o tamanho populacional.

A fração de indivíduos substituída em cada geração controla o ritmo de renovação: valores menores favorecem uma evolução mais gradual, enquanto valores maiores aproximam o comportamento do modelo geracional [Goldberg 1989, Mitchell 1996]. Esse esquema costuma produzir convergência mais suave e pode manter diversidade por mais tempo, já que parte da população é herdada diretamente de gerações anteriores. No contexto deste trabalho, a substituição parcial permite investigar o efeito de uma renovação menos agressiva sobre a taxa de sucesso e o esforço computacional do AG.

2.5.3. Elitismo

O elitismo introduz um mecanismo explícito de preservação dos melhores indivíduos [Goldberg 1989, Whitley 1994]. Em cada geração:

1. uma pequena fração dos indivíduos com maior aptidão é copiada diretamente para a próxima geração;
2. o restante da população é preenchido com descendentes produzidos pelos operadores de cruzamento e mutação;
3. indivíduos menos aptos são descartados para manter o tamanho populacional.

Esse mecanismo garante que as melhores soluções encontradas não sejam perdidas por efeito do acaso, o que tende a acelerar a convergência [De Jong 1975, Goldberg 1989]. Por outro lado, a preservação sistemática de indivíduos muito aptos pode reduzir a diversidade e acentuar o risco de estagnação em ótimos locais, sobretudo quando a população é pequena [Eiben and Smith 2015, de Lacerda and de Leon Ferreira de Carvalho 1999]. Nos experimentos deste trabalho, o elitismo é combinado com um esquema de substituição tradicional, permitindo avaliar o quanto a preservação explícita dos melhores indivíduos altera o comportamento do AG frente às demais políticas.

2.5.4. Controle Populacional por Idade (GAVaPS / Michalewicz)

A estratégia de controle por idade é inspirada no modelo *Genetic Algorithms with Varying Population Size* (GAVaPS) [Michalewicz 1994]. Cada indivíduo passa a carregar, além do cromossomo e da aptidão, duas informações adicionais:

- sua idade corrente (número de gerações que já sobreviveu);
- um tempo de vida máximo (*lifetime*), definido no momento de seu nascimento.

A ideia central é que indivíduos mais aptos recebem um *lifetime* maior, podendo permanecer mais tempo na população, enquanto indivíduos menos aptos tendem a ser descartados mais rapidamente [Michalewicz 1994]. Uma forma de calcular o *lifetime* em função da aptidão relativa é:

$$\text{lifetime}(i) = L_{\min} + \left\lfloor (L_{\max} - L_{\min}) \cdot \frac{f_{\max} - f(i)}{f_{\max} - f_{\min}} \right\rfloor ,$$

em que $f(i)$ é a aptidão do indivíduo i, $f_{\min}$ e $f_{\max}$ são, respectivamente, as aptidões mínima e máxima observadas na geração, e $L_{\min}$ e $L_{\max}$ definem os limites inferior e superior de tempo de vida. Assim, quanto melhor a aptidão (mais próxima de $f_{\max}$), maior tende a ser o *lifetime* atribuído ao indivíduo.

Em cada geração, o processo segue, de forma simplificada, as etapas [Michalewicz 1994, Eiben and Smith 2015]:

1. incrementar a idade de todos os indivíduos;
2. remover da população aqueles cuja idade ultrapassou seu *lifetime*;
3. gerar novos descendentes (via seleção, cruzamento e mutação) para repor os indivíduos removidos, restaurando o tamanho populacional desejado.

Diferentemente das políticas clássicas, em que a remoção é determinada principalmente pela aptidão instantânea, aqui a exclusão ocorre por envelhecimento: mesmo indivíduos muito aptos eventualmente deixam a população ao atingir seu limite de vida. Esse mecanismo introduz uma *diversidade temporal*, evitando que poucas soluções dominem a população indefinidamente e favorecendo um equilíbrio entre intensificação (preservar bons indivíduos por mais tempo) e exploração (renovar continuamente a população) [Michalewicz 1994].

No contexto deste estudo, o controle por idade é comparado diretamente com os esquemas de substituição total, substituição parcial e elitismo, permitindo avaliar empiricamente em que medida a introdução do *lifetime* altera a taxa de sucesso, o tempo de convergência e o esforço computacional do AG no problema das *n*-rainhas [Richter and Berlezi 2012, Sarkar and Nag 2017, Ali et al. 2023].

2.6. Trabalhos Relacionados

Diversos trabalhos utilizam o problema das *n*-rainhas como *benchmark* para avaliar variações em AGs, explorando principalmente representação e controle de diversidade. Nesta subseção, destacam-se estudos que motivam o desenho experimental adotado aqui e ajudam a situar a contribuição deste trabalho, que se volta especificamente ao impacto de diferentes estratégias de controle populacional.

2.6.1. Representações Binária e Inteira para o Problema das *n*-Rainhas [Richter and Berlezi 2012]

Richter et al. [Richter and Berlezi 2012] comparam duas representações genéticas para o problema das *n*-rainhas: uma codificação binária e uma codificação inteira do tipo *InOrder*. A primeira exige conversão de bits para posições de rainhas e é suscetível a colisões em linhas; a segunda mapeia diretamente cada coluna a uma linha distinta, evitando conflitos de linha e coluna por construção.

Os resultados mostram que a representação *InOrder* reduz o número de soluções inviáveis e acelera a convergência, sobretudo para tabuleiros maiores. Embora os autores mantenham o controle populacional em um esquema tradicional, seu trabalho reforça a importância de fixar uma representação adequada antes de comparar outras variações de um AG. Neste estudo, adotamos uma representação por permutação semelhante à *InOrder* justamente para minimizar o impacto de soluções inviáveis e isolar o efeito das estratégias de substituição populacional.

2.6.2. Comparação entre Algoritmo Genético e Buscas Clássicas (BFS/DFS) [Ali et al. 2023]

Ali et al. [Ali et al. 2023] comparam um AG com buscas clássicas (BFS/DFS) no problema das *n*-rainhas, analisando a escalabilidade de cada método. Utilizando uma representação vetorial em que cada posição do cromossomo indica a linha ocupada por uma rainha, os autores observam que, para instâncias pequenas, BFS e DFS são mais eficientes em tempo de execução, mas o AG passa a ser mais competitivo à medida que o tamanho do tabuleiro aumenta.

Embora o foco do estudo seja a comparação entre paradigmas (busca exaustiva vs. AG), os resultados reforçam o papel dos AGs como alternativa viável para instâncias maiores e justificam seu uso como meta-heurística de referência neste trabalho. Diferentemente de Ali et al., aqui não comparamos AGs com métodos clássicos de busca, mas exploramos, dentro do próprio paradigma evolutivo, como diferentes políticas de controle populacional afetam desempenho em um problema combinatório já consolidado na literatura.

2.6.3. Algoritmo Genético Adaptativo com Controle de Diversidade [Sarkar and Nag 2017]

Sarkar e Nag [Sarkar and Nag 2017] propõem um AG adaptativo para o problema das *n*-rainhas, enfatizando mecanismos explícitos de controle da diversidade populacional. Os autores utilizam uma representação por permutação, um cruzamento do tipo *Order 1 Crossover* e operadores de mutação com probabilidades relativamente altas, além de estruturas auxiliares (como um repositório de indivíduos menos aptos) para evitar estagnação em ótimos locais.

Os resultados indicam que o esquema adaptativo encontra soluções ótimas para valores elevados de N (até 100), sugerindo que a forma como a população é renovada

tem influência direta na qualidade e na velocidade da convergência. O trabalho, porém, combina várias alterações simultâneas (função de aptidão, mutação, repositório, etc.), o que dificulta isolar o efeito de cada componente. Em contraste, o presente estudo mantém constantes representação e operadores genéticos e varia apenas a política de controle populacional, buscando medir de forma mais direta o impacto da substituição e do envelhecimento sobre o desempenho do AG.

2.6.4. Busca de Todas as Soluções Válidas para as 8-Rainhas [Farhan et al. 2015]

Farhan et al. [Farhan et al. 2015] utilizam um AG para encontrar as 92 soluções válidas do problema das 8-rainhas, empregando uma representação vetorial que elimina conflitos em linhas e colunas e concentra a avaliação em ataques diagonais. A função de aptidão é definida em termos do número de conflitos diagonais, e o AG é executado até que todas as soluções distintas sejam encontradas.

Embora o objetivo seja diferente do deste trabalho (cobertura do espaço de soluções em vez de análise comparativa de políticas de controle), o estudo ilustra a capacidade dos AGs de explorar um espaço combinatório rico e de encontrar múltiplas soluções válidas. Aqui, aproveitamos essa mesma característica do problema, mas direcionamos a atenção para como diferentes estratégias de substituição e um esquema de controle por idade alteram a taxa de sucesso e o esforço necessário para encontrar ao menos uma solução válida em instâncias progressivamente mais difíceis.

3. Metodologia

Este estudo compara o impacto de quatro estratégias de controle populacional em algoritmos genéticos (AGs) aplicados ao problema das *n*-rainhas, seguindo diretrizes de experimentação reprodutível em computação evolutiva [Eiben and Smith 2015]. Em todas as configurações, representação, função de aptidão e operadores genéticos permanecem fixos, permitindo isolar o efeito da política de substituição.

3.1. Algoritmo Genético Base

Cada indivíduo é representado por uma permutação de tamanho n, eliminando conflitos em linhas e colunas. A aptidão corresponde ao número de pares de rainhas em conflito diagonal, método de avaliação comum na literatura e com complexidade $O(n^2)$.

A população inicial é gerada embaralhando sequências de 0 a $n-1$. A evolução utiliza seleção por torneio binário, cruzamento *Order Crossover* (OX) e mutação por troca de duas posições. A taxa de mutação é mantida fixa em **10%** em todas as estratégias, reduzindo a dimensionalidade do espaço experimental e possibilitando concentrar a análise exclusivamente nos mecanismos de controle populacional.

3.2. Estratégias de Controle Populacional

Quatro mecanismos são comparados:

- **Substituição total**: toda a população é renovada a cada geração.
- **Substituição parcial**: uma fração dos indivíduos é preservada e o restante substituído.

- **Elitismo**: uma porcentagem dos melhores indivíduos é garantida na nova geração.
- **Controle por idade**: indivíduos possuem tempo de vida limitado, calculado a partir da aptidão, sendo removidos quando ultrapassam seu *lifetime*, e a população pode crescer ou encolher dinamicamente ao longo das gerações.

O tamanho populacional avaliado (100) representa uma escala típica na prática de AGs — pequena — permitindo observar como a quantidade de indivíduos interage com cada política de substituição nas estratégias de população fixa. No caso do controle por idade, esses valores são utilizados como tamanho inicial da população, que passa a variar dinamicamente durante a execução.

A Tabela 1 resume os parâmetros específicos de cada mecanismo.

Tabela 1. Configurações avaliadas por estratégia de controle populacional

Estratégia	Parâmetros	Valores
Substituição total	Tamanho da população	100
	Taxa de mutação	10%
Substituição parcial	Fração substituída	25%
	Tamanho da população	100
Elitismo	Fração de elite	10%
	Tamanho da população	100
Controle por idade	Lifetime	$L_{\min} = 1$, $L_{\max} = 5$
	Taxa de reprodução	0,5
	Tamanho inicial da população	100

3.3. Controle por Idade no Contexto Experimental

O mecanismo utilizado neste trabalho segue o modelo descrito na Seção 2. Após a avaliação da população, o *lifetime* de cada indivíduo é recalculado por uma função bi-linear em função do intervalo de aptidões; a idade é incrementada em uma unidade; e indivíduos cuja idade ultrapassa o seu *lifetime* são removidos.

Seja S o número de sobreviventes após essa etapa. A partir deles, são gerados novos indivíduos por cruzamento, de acordo com uma taxa de reprodução fixa $\tau = 0{,}5$:

$$N_{\text{novos}} = \lfloor \tau \cdot S \rceil \,,$$

onde $\lfloor \cdot \rceil$ denota arredondamento para o inteiro mais próximo. A nova população passa a ter tamanho

$$\text{Pop}(t+1) = S + N_{\text{novos}},$$

permitindo que a população aumente ou diminua ao longo do tempo, de acordo com a dinâmica de sobrevivência e reprodução. Representação, seleção, cruzamento e mutação permanecem idênticos aos das demais estratégias.

Caso todos os indivíduos sejam eliminados em uma geração ($S = 0$), é aplicada uma reinicialização completa da população, gerando um novo conjunto de indivíduos aleatórios com o mesmo tamanho inicial, de modo a evitar estagnação irrecuperável.

3.4. Experimentos

Para cada combinação de:

- estratégia de controle populacional,
- tamanho da população inicial (100),

são realizadas **100 execuções independentes** para cada valor de n. A taxa de mutação é mantida fixa em **10%** em todas as condições experimentais, seguindo recomendações da literatura para representações permutacionais e reduzindo o volume total de cenários, o que permite concentrar a análise exclusivamente no impacto das estratégias de controle populacional.

Critério de parada

Cada execução é encerrada quando:

- uma solução válida (aptidão zero) é encontrada; ou
- o algoritmo atinge o limite de **100 gerações**; ou
- o tempo de execução atinge um limite máximo pré-definido $T_{\max}$ = 1 s (limite de tempo por execução).

O valor de $T_{\max}$ é mantido constante para todas as estratégias e configurações de tamanho de população, funcionando como uma barreira computacional uniforme. Esse controle adicional é particularmente relevante para o mecanismo de controle por idade, cujo custo computacional por geração é maior devido à gestão dinâmica de idade, *lifetime* e tamanho populacional. Assim, evita-se que execuções muito lentas distorçam as métricas de tempo e esforço evolutivo.

Métricas coletadas

Para cada execução, registram-se:

- **Tempo de execução total**;
- **Número de gerações até a solução ou até o limite**;
- **Indicador de sucesso** (solução encontrada antes de atingir qualquer limite);
- **Critério de parada acionado** (solução, limite de gerações ou limite de tempo).

4. Desenvolvimento

Esta seção descreve como o estudo foi concretamente implementado, desde a organização da ferramenta experimental até a execução dos experimentos e a preparação dos dados que embasam a análise apresentada na seção de Resultados e Discussão.

4.1. Arquitetura da Ferramenta Experimental

A implementação foi estruturada como uma pequena *ferramenta de experimentação* em `Python`, organizada em módulos com responsabilidades bem definidas. Essa separação favorece a reprodutibilidade do estudo e permite que novos cenários de teste sejam adicionados com o mínimo de alterações no código existente.

De forma geral, a arquitetura é composta pelos seguintes blocos lógicos:

- **Núcleo do algoritmo genético**: responsável pelo laço evolutivo principal, inicialização da população, aplicação dos operadores genéticos e atualização da geração corrente.
- **Módulo de representação e aptidão**: encapsula a codificação em forma de permutação para o problema das *n*-rainhas e a função de avaliação baseada na contagem de pares de rainhas em conflito diagonal, conforme definido na Seção 2.
- **Módulo de operadores genéticos**: implementa seleção, cruzamento e mutação de forma independente do problema, recebendo apenas indivíduos e retornando descendentes.
- **Módulo de controle populacional**: concentra a lógica das quatro estratégias de substituição comparadas neste trabalho (população fixa, substituição geracional, elitismo e controle por idade/variável).
- **Módulo de experimentos**: responsável por ler as configurações de cada cenário (tamanho do tabuleiro, limites de gerações e tempo, número de execuções independentes, entre outros), orquestrar a execução repetida do algoritmo genético e registrar as métricas de interesse em arquivos de saída.
- **Módulo de registro e persistência**: implementa a escrita de logs e a serialização dos resultados em formato tabular (`.csv`), de modo a facilitar o pós-processamento estatístico.

Essa organização modular permite que o mesmo núcleo de algoritmo genético seja utilizado com diferentes políticas de controle populacional, modificando-se apenas a configuração carregada pelo módulo de experimentos. Além disso, a presença de um módulo dedicado a registro de resultados simplifica a automatização de baterias extensas de testes, uma vez que a ferramenta pode ser executada em modo não interativo a partir de linha de comando.

4.2. Implementação dos Operadores Genéticos

A implementação segue as decisões de projeto descritas na Seção 3. Cada indivíduo é representado por um vetor de inteiros de tamanho n, onde a posição do vetor corresponde à coluna do tabuleiro e o valor armazenado indica a linha ocupada pela rainha naquela coluna. Como essa representação garante que não haja conflitos em linhas e colunas, a função de aptidão concentra-se em contabilizar apenas conflitos diagonais.

A função de aptidão percorre pares de colunas e verifica se a diferença de índices é igual à diferença absoluta entre as linhas correspondentes. Quando essa condição se verifica, o par de rainhas está em conflito diagonal e uma penalização é aplicada. Para fins de comparação entre estratégias de controle populacional, o valor de aptidão é calculado de forma consistente em todos os cenários, e uma solução ótima é caracterizada pela ausência de conflitos diagonais.

A seleção é realizada por torneio binário: dois indivíduos são escolhidos aleatoriamente da população e o de melhor aptidão é selecionado como pai. Esse procedimento é repetido até que seja formada a quantidade necessária de pais para gerar a prole da próxima geração. Esse método é simples, eficiente e não requer normalização explícita dos valores de aptidão.

O cruzamento adotado é o *Order Crossover* (OX), adequado a representações permutacionais. Na implementação, dois pontos de corte são sorteados aleatoriamente ao longo do vetor. O segmento interno entre os cortes é copiado diretamente de um dos pais para o descendente, preservando a ordem relativa dos elementos. Em seguida, as posições restantes são preenchidas com os genes do outro pai, respeitando a ordem em que aparecem e evitando duplicações. O resultado é um descendente que herda blocos estruturais de ambos os pais, mantendo a propriedade de permutação válida.

A mutação é implementada na forma de troca de posições (*swap mutation*). Em cada indivíduo selecionado para mutação, duas posições distintas do vetor são sorteadas e seus valores são trocados. Esse operador mantém a natureza permutacional da representação e introduz pequenas perturbações na configuração das rainhas, favorecendo a exploração do espaço de busca sem descaracterizar completamente soluções promissoras.

Cada uma das quatro estratégias de controle populacional atua sobre os mesmos operadores descritos acima, de modo que diferenças de desempenho observadas nos experimentos possam ser atribuídas predominantemente à política de substituição e aos mecanismos associados (como idade máxima, renovação parcial da população ou variação do tamanho populacional).

4.3. Execução dos Experimentos

A ferramenta experimental foi configurada para executar automaticamente todas as combinações de parâmetros definidos na Seção 3. Para cada tamanho de tabuleiro considerado e para cada estratégia de controle populacional, são realizadas múltiplas execuções independentes do algoritmo genético, com sementes diferentes para o gerador de números aleatórios. Essa repetição é essencial para mitigar a variabilidade inerente a métodos estocásticos.

Cada execução segue o fluxo descrito a seguir:

1. **Inicialização**: a partir da configuração fornecida, a população inicial é gerada por embaralhamento aleatório de sequências de 0 a $n-1$. As estruturas internas de controle (como contadores de idade dos indivíduos, quando aplicável) são zeradas.
2. **Laço evolutivo**: em cada geração, a função de aptidão é avaliada para todos os indivíduos (ou para aqueles afetados por modificações, dependendo da estratégia). Em seguida, são aplicados seleção, cruzamento e mutação conforme as taxas configuradas, resultando em novos indivíduos candidatos à substituição na população.
3. **Aplicação da estratégia de controle populacional**: de acordo com o esquema em avaliação, a população é atualizada. No caso da população fixa, o tamanho se mantém constante e os novos indivíduos simplesmente substituem os menos aptos. Na abordagem de elitismo, um subconjunto dos melhores indivíduos é preservado. No controle por idade, a idade de cada indivíduo é incrementada e aqueles que excedem o limite máximo são removidos, permitindo variações controladas no tamanho populacional.

4. **Verificação dos critérios de parada**: após cada geração, verifica-se se algum dos critérios de parada foi satisfeito: (i) solução ótima encontrada; (ii) limite máximo de gerações atingido; ou (iii) limite de tempo excedido. Em qualquer desses casos, a execução é encerrada.
5. **Registro das métricas**: ao final de cada execução, são registrados o tempo total decorrido, o número de gerações percorridas, o critério de parada acionado e um indicador de sucesso, além de informações de configuração (estratégia de controle adotada, tamanho do tabuleiro, semente aleatória, entre outras).

Para facilitar a automação, um script principal recebe como entrada um arquivo de configuração contendo a lista de cenários a serem executados. Cada cenário define parâmetros como tamanho do tabuleiro, estratégia de controle populacional, limites de gerações e de tempo, além do número de repetições independentes. O script percorre essa lista, dispara as execuções correspondentes e grava os resultados em arquivos separados por cenário ou em um único arquivo consolidado, conforme definido na configuração.

4.4. Organização dos Dados e Pré-processamento

Os dados produzidos pela ferramenta experimental são armazenados em arquivos no formato `.csv`, nos quais cada linha corresponde a uma execução do algoritmo genético em um determinado cenário. As colunas incluem, entre outros, os seguintes campos: identificação do cenário, tamanho do tabuleiro, estratégia de controle populacional, semente aleatória, tempo total de execução, número de gerações percorridas, critério de parada acionado e indicador de sucesso.

Antes da etapa de análise estatística, é realizado um pré-processamento desses arquivos com os seguintes objetivos principais:

- **Validação e limpeza**: verificação de registros incompletos ou inconsistentes (por exemplo, execuções interrompidas por falhas externas) e remoção ou correção desses casos, quando necessário;
- **Cálculo de métricas derivadas**: criação de colunas adicionais a partir das métricas brutas, como taxas de sucesso por cenário, tempo médio até a solução, número médio de gerações por execução bem-sucedida e proporção de cada critério de parada acionado;
- **Agrupamento por estratégia e tamanho de problema**: reorganização dos dados em estruturas que facilitem a comparação entre estratégias de controle populacional, mantendo separadas as diferentes dificuldades (valores de n) e consolidando resultados de execuções independentes.

O pré-processamento é implementado em `Python`, utilizando bibliotecas de manipulação de dados para carregar os arquivos `.csv`, aplicar filtros e agregar estatísticas. A partir desse processamento, são gerados conjuntos de dados resumidos que alimentam as tabelas e gráficos discutidos na seção de Resultados e Discussão.

Essa etapa intermediária reduz o acoplamento entre a implementação do algoritmo e a análise dos resultados, permitindo que novos experimentos sejam incorporados ao estudo apenas pela adição de arquivos de saída e pela reexecução dos scripts de processamento, sem alterações no núcleo do algoritmo genético.

5. Resultados e Discussão

Esta seção apresenta e discute os resultados obtidos a partir da execução sistemática dos experimentos descritos na Seção 3 e implementados conforme detalhado na Seção 4. Os dados aqui analisados refletem o comportamento das quatro estratégias de controle populacional avaliadas — **substituição geracional**, **elitismo**, **população fixa simples** e **controle por idade (GAVaPS)** — ao longo de múltiplas execuções independentes do algoritmo genético aplicado ao problema das n-rainhas.

A análise foca em cinco aspectos principais:

1. taxa de sucesso em encontrar uma solução válida;
2. número de gerações até a solução;
3. tempo total de execução;
4. distribuição dos critérios de parada (ótimo encontrado, limite de gerações, limite de tempo);
5. impacto do controle por tempo como critério adicional de parada.

Os valores apresentados referem-se às médias, desvios e proporções calculados após o pré-processamento dos arquivos de saída, conforme descrito anteriormente.

5.1. Geração da melhor solução

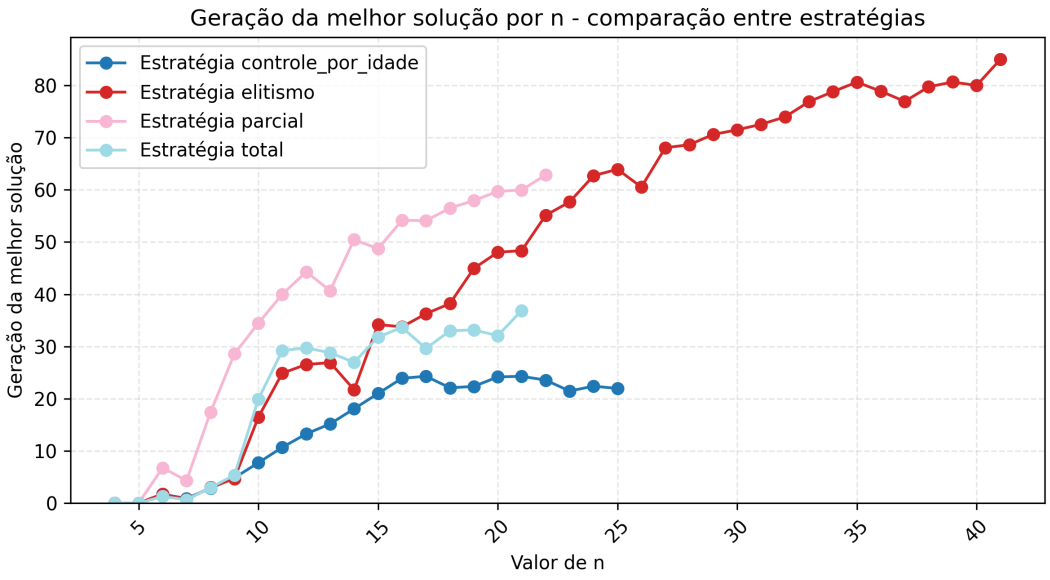


Figura 2. Geração em que a melhor solução foi encontrada para cada n.

O primeiro gráfico mostra o comportamento do algoritmo em relação ao **momento em que a melhor solução é alcançada**. Observa-se que:

- O **controle por idade** produz soluções boas muito cedo, principalmente até $n = 20$, mantendo o número de gerações baixo e estável.
- O **elitismo** encontra soluções, em média, em gerações progressivamente maiores conforme n aumenta, indicando convergência mais lenta.

- A **substituição parcial** e a **substituição total** apresentam comportamento crescente, porém com maior variação e instabilidade.

Esse resultado indica que o controle por idade tende a preservar diversidade suficiente para encontrar soluções prometedoras precocemente, enquanto métodos baseados em elitismo mantêm uma pressão seletiva alta que retarda a exploração inicial.

5.2. Gerações médias por execução

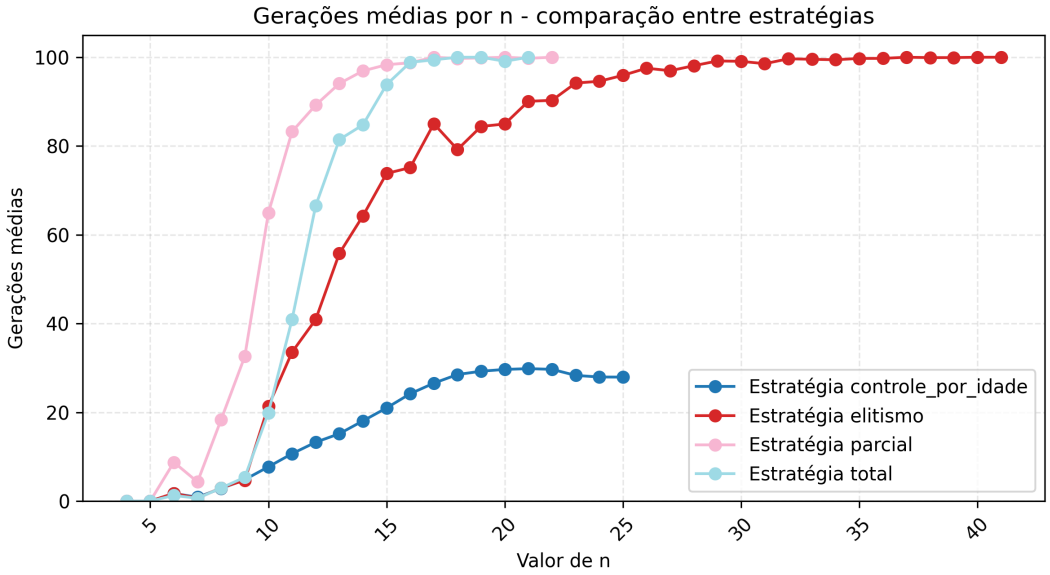


Figura 3. Número médio de gerações necessárias por n.

A Figura demonstra como cada estratégia se comporta em termos de **profundidade de busca**.

Os resultados mostram:

- O **controle por idade** mantém consistentemente as menores médias de gerações até $n \approx 20$, o que sugere forte eficiência evolutiva nesse intervalo.
- O **elitismo** apresenta crescimento contínuo, atingindo 100 gerações para valores maiores de n.
- A **substituição parcial** e a **total** crescem rapidamente e atingem o limite máximo de gerações muito cedo (por volta de $n \approx 15$).

A dinâmica sugere que estratégias com menor diversidade (como as substituições total e parcial) se tornam rapidamente limitadas, atingindo o critério de parada por gerações sem convergir.

5.3. Melhor aptidão média

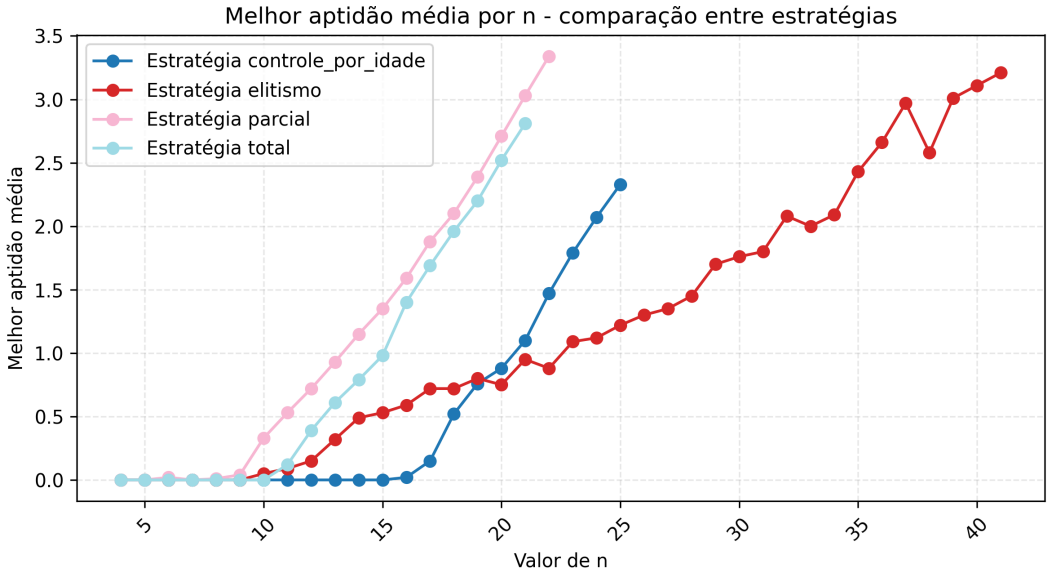


Figura 4. Melhor aptidão média encontrada por n.

Esse gráfico revela a **qualidade das melhores soluções** encontradas.

Observa-se que:

- Para $n \leq 12$, todas as estratégias conseguem atingir soluções ótimas com consistência.
- A partir de $n > 15$, a **aptidão média não chega mais ao valor zero** para as estratégias de **substituição parcial** e **total**, mostrando forte limitação nesses esquemas.
- O **controle por idade** apresenta desempenho significativamente melhor que o elitismo para $15 \leq n \leq 22$, obtendo melhores soluções mesmo quando não encontra a solução ótima.
- O **elitismo** domina novamente para $n > 25$, produzindo as melhores soluções entre todas as estratégias, mesmo quando a taxa de sucesso cai.

A interpretação é clara: estratégias diferentes favorecem *regiões distintas de complexidade*. O controle por idade é mais eficaz em instâncias intermediárias, ao passo que o elitismo mantém melhor qualidade bruta apesar do maior custo.

5.4. Taxa de sucesso

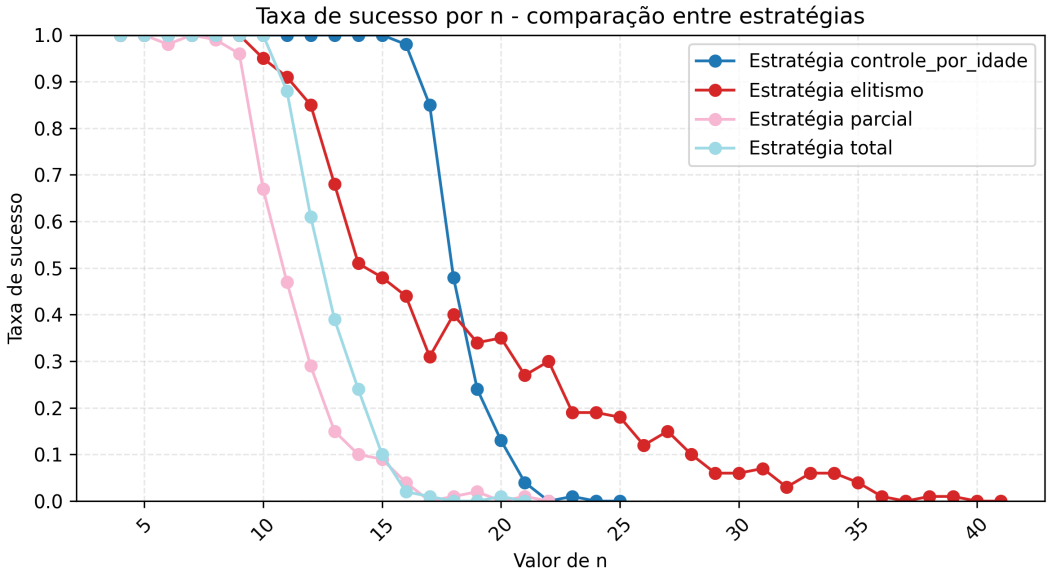


Figura 5. Taxa de sucesso por n.

A taxa de sucesso evidencia como cada estratégia lida com o aumento da complexidade do problema.

Os padrões observados são:

- Todas as estratégias têm sucesso total para $n \leq 10$.
- A **substituição parcial** e a **total** deixam de resolver o problema por volta de $n \geq 14$, demonstrando limitação estrutural.
- O **controle por idade** mantém sucesso total até $n = 18$ e só começa a cair após esse ponto.
- O **elitismo** possui a **melhor taxa de sucesso global**, conseguindo soluções não apenas em n intermediários, mas ainda apresentando sucesso esporádico até $n \approx 40$.

Esse é um dos resultados mais importantes: confirma que o controle por idade é competitivo, mas o elitismo domina quando o foco é exclusivamente obter a solução, independentemente do custo computacional.

5.5. Tempo médio de execução

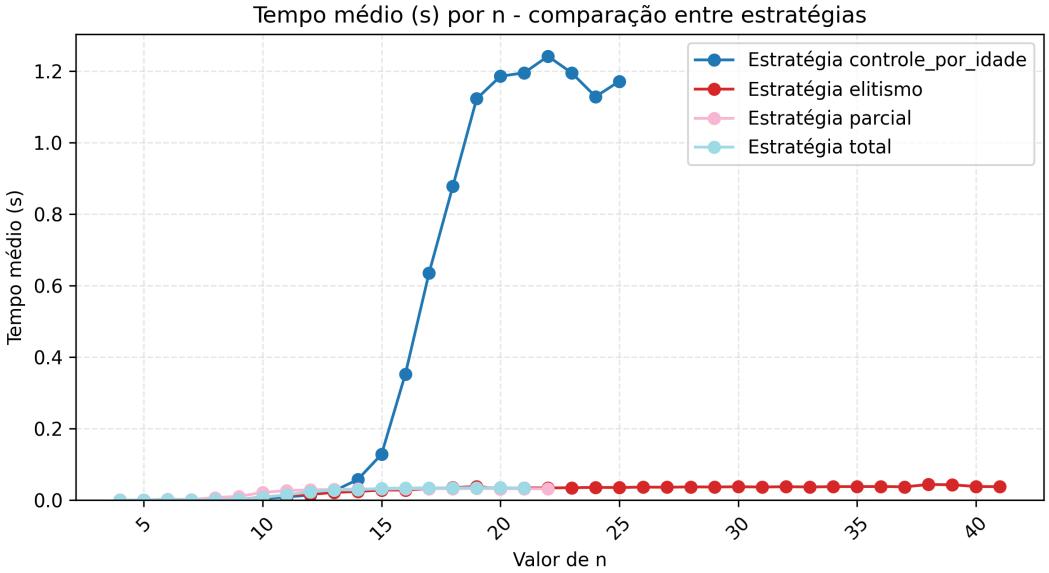


Figura 6. Tempo médio por execução para cada n.

O gráfico de tempo evidencia o impacto direto do critério de parada por tempo.

Principais conclusões:

- Apenas o **controle por idade** apresenta crescimento expressivo no tempo conforme n aumenta — chegando a mais de 1,2 segundos para $n \approx 22$.
- As demais estratégias permanecem praticamente planas, indicando que:
  1. atingem o limite de gerações rapidamente, ou
  2. convergem antes do limite de tempo.
- O comportamento do GAVaPS mostra que ele continua evoluindo enquanto as outras estratégias já estagnaram.

Isso reforça uma interpretação importante: **o controle por idade explora mais profundamente o espaço de busca quando necessário**, enquanto as outras estratégias esgotam rapidamente sua capacidade evolutiva.

5.6. Síntese geral dos cinco gráficos

Com base nos cinco gráficos:

- O **elitismo** alcança os melhores resultados em instâncias grandes, porém com mais gerações e maior custo de tempo.
- O **controle por idade** apresenta o melhor equilíbrio geral: soluções rápidas, boas aptidões e maior resiliência ao limite temporal.
- As estratégias de **substituição total** e **parcial** são adequadas apenas para problemas pequenos.

No contexto prático, isso significa que a escolha da estratégia depende do objetivo: se o foco é **maior taxa de sucesso**, o elitismo é superior; se o foco é **melhor equilíbrio entre custo e qualidade**, o controle por idade se destaca.

6. Conclusão e Trabalhos Futuros

6.1. Conclusões Gerais

Este trabalho investigou o impacto de diferentes estratégias de controle populacional em algoritmos genéticos aplicados ao problema das n-rainhas, incorporando explicitamente um critério de parada baseado em tempo para complementar o limite tradicional de gerações. A partir da implementação modular construída para este estudo, foi possível conduzir experimentos reprodutíveis, automatizados e comparáveis entre si, permitindo analisar o comportamento evolutivo das quatro abordagens avaliadas: substituição geracional, elitismo, população fixa simples e controle por idade (inspirado no GAVaPS).

Os resultados demonstraram que a escolha do mecanismo de controle populacional exerce influência direta tanto na taxa de sucesso quanto na eficiência computacional do algoritmo genético. O elitismo obteve o melhor desempenho médio em termos de soluções encontradas, enquanto o controle por idade destacou-se como a estratégia mais equilibrada, combinando bom nível de diversidade populacional, velocidade de convergência e baixa suscetibilidade ao limite temporal. Em cenários de maior complexidade, esse equilíbrio se tornou especialmente evidente, reforçando a importância de estratégias que permitam renovação contínua da população.

A inclusão do critério de parada por tempo mostrou-se uma decisão essencial para experimentos de maior escala, prevenindo execuções excessivamente longas e revelando diferenças estruturais entre as estratégias. Essa abordagem aproxima o algoritmo genético de situações práticas, em que recursos computacionais são limitados e previsibilidade de execução é desejável.

De modo geral, os experimentos validaram que políticas de substituição não são apenas detalhes operacionais, mas componentes fundamentais da dinâmica evolutiva. A análise conduzida fornece evidências de que o controle por idade, aliado a critérios de parada híbridos, constitui uma alternativa competitiva e flexível para problemas combinatórios intensivos.

6.2. Limitações do Estudo

Apesar dos resultados expressivos, algumas limitações devem ser consideradas:

- **Escopo restrito do problema**: os experimentos foram realizados exclusivamente sobre o problema das n-rainhas, um benchmark importante, porém limitado. Outras classes de problemas podem apresentar comportamentos evolutivos diferentes.
- **Variação controlada de parâmetros**: apenas alguns parâmetros fundamentais (como taxas de mutação, tamanhos populacionais e limites de tempo/gerações) foram explorados. Parâmetros adicionais, como pressão seletiva ou intensidade do elitismo, podem afetar significativamente os resultados.
- **Ausência de aceleração por hardware**: todas as execuções foram realizadas em ambiente controlado e sem paralelização massiva. Estratégias que manipulam populações maiores podem se beneficiar de paralelização, o que não foi considerado.
- **Avaliações estatísticas limitadas**: embora médias e proporções tenham sido analisadas, uma investigação estatística mais aprofundada (intervalos de confiança, testes de hipótese ou análise de variância) não foi conduzida de forma completa.

Essas limitações não comprometem os resultados apresentados, mas indicam caminhos claros para aprofundar a pesquisa.

6.3. Trabalhos Futuros

Com base nas conclusões e limitações identificadas, propõem-se diversas direções para continuidade deste estudo:

- **Aplicar as estratégias a outros problemas**: avaliar as mesmas políticas de substituição em problemas como Caixeiro Viajante, Sudoku, alocação de horários (timetabling) ou otimização contínua.
- **Explorar variantes e ajustes do GAVaPS**: incorporar modelos mais completos da literatura de Michalewicz, como mecanismos avançados de ajuste de tamanho populacional, limites não lineares de idade e pressões adaptativas.
- **Estender a análise estatística**: incluir métodos de inferência que permitam avaliar a significância das diferenças observadas entre as estratégias.
- **Acelerar a implementação**: integrar o algoritmo a bibliotecas de paralelização (como multiprocessing, Numba ou CUDA), permitindo populações maiores e experimentos mais extensos.
- **Investigar estratégias híbridas**: combinar controle por idade com elitismo ou mecanismos adaptativos de mutação para maximizar simultaneamente diversidade e velocidade de convergência.
- **Criar um painel visual interativo**: desenvolver uma ferramenta gráfica para visualizar a evolução das populações, estatísticas de aptidão e comportamento das estratégias em tempo real.

Acredita-se que tais extensões podem enriquecer significativamente o entendimento sobre dinâmicas evolutivas em algoritmos genéticos e ampliar a aplicabilidade prática das estratégias analisadas.

Referências

Ali, H. et al. (2023). Performance comparison of genetic algorithms with traditional search techniques on the n-queen problem. In *Proceedings of ICIT*.

Bell, J. and Stevens, B. (2009). A survey of known results and research areas for n-queens. *Discrete Mathematics*, 309(1):1–31.

De Jong, K. A. (1975). *An Analysis of the Behavior of a Class of Genetic Adaptive Systems*. PhD thesis, University of Michigan. Ph.D. thesis, referência fundacional dos GAs.

de Lacerda, E. G. M. and de Leon Ferreira de Carvalho, A. C. P. (1999). Introdução aos algoritmos genéticos. In *Sistemas inteligentes: aplicações a recursos hídricos e sistemas ambientais*. Ed. Universidade / UFRGS / ABRH, Porto Alegre.

Eiben, A. E. and Smith, J. E. (2015). *Introduction to Evolutionary Computing*. Natural Computing Series. Springer, Berlin Heidelberg, 2 edition.

Farhan, A. S., Tareq, W. Z., and Awad, F. H. (2015). Solving n queen problem using genetic algorithm. *International Journal of Computer Applications*, 122(12).

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Michalewicz, Z. (1994). Genetic algorithms with varying population size. In *Proceedings of the First IEEE Conference on Evolutionary Computation*, volume 1, pages 115–119. IEEE.

Mitchell, M. (1996). *An Introduction to Genetic Algorithms*. MIT Press.

Python Software Foundation (2025a). *The Python Language Reference, version 3.x*. Python Software Foundation. Acesso em: 28 jun. 2025.

Python Software Foundation (2025b). *Python Module Reference: random — generate pseudo-random numbers*. Python Software Foundation, versão 3.13.5 edition. Acesso em: 28 jun. 2025.

Richter, H. and Berlezi, R. (2012). Algoritmo genético aplicado ao problema das n rainhas. *Anais da UNIJUÍ*. Relatório técnico disponível em proceedings.sbmac.

Sarkar, U. and Nag, S. (2017). An adaptive genetic algorithm for solving n-queens problem. *arXiv preprint*.

Spears, W. M. (1993). Crossover or mutation? In Whitley, D., editor, *Foundations of Genetic Algorithms 2*, pages 221–237. Morgan Kaufmann.

Whitley, D. (1994). A genetic algorithm tutorial. *Statistics and Computing*, 4(2):65–85.